Epic 5: Merchant Dashboard v1

CONV-51 through CONV-55

Conversiono Course Documentation

Stack: Next.js 14 | React 18 | TypeScript | Tailwind CSS v4 | Recharts

Author: AI-assisted development | Date: March 2026

Chapter 1: Architecture Overview

The Merchant Dashboard is a Next.js 14 App Router application that provides a real-time analytics interface for e-commerce merchants using Conversiono. It connects to the FastAPI backend API built in Epics 3-4 and visualizes session data, behavioral analytics, experiments, and intervention performance.

Technology Choices

- Next.js 14 App Router: File-system routing, React Server Components support, built-in optimization
- React 18: Concurrent features, hooks-based architecture
- TypeScript: Full type safety across all components and API calls
- Tailwind CSS v4: Utility-first styling with @tailwindcss/postcss plugin
- Recharts: Composable charting library for data visualization
- clsx: Conditional CSS class merging utility

Directory Structure

```
frontend/src/
  app/
    layout.tsx              # Root layout with Tailwind globals
    page.tsx                # Redirect to /dashboard
    globals.css             # Tailwind v4 import
    dashboard/
      layout.tsx            # Dashboard shell wrapper
      page.tsx              # Overview (KPIs, funnel, risk)
      sessions/
        page.tsx            # Session list with filters
        [id]/page.tsx       # Session detail view
      analytics/page.tsx    # Full analytics dashboard
      experiments/page.tsx  # Experiment CRUD + stats
      interventions/page.tsx # Intervention performance
      settings/page.tsx     # SDK key + merchant profile
  components/
    layout/
      Sidebar.tsx           # Navigation sidebar
      DashboardShell.tsx    # Shell with responsive sidebar
    ui/
      Card.tsx              # Card, CardHeader, CardBody
      StatCard.tsx          # KPI metric card
      Badge.tsx             # Status badge
      Spinner.tsx           # Loading spinner
      ErrorBox.tsx          # Error display with retry
      EmptyState.tsx        # Empty state placeholder
  lib/
    api.ts                  # Typed API client (26 endpoints)
    hooks.ts                # useQuery custom hook
    format.ts               # Formatting utilities
```

Chapter 2: API Client Layer (CONV-51)

The API client in src/lib/api.ts is a centralized, typed HTTP client that handles all communication with the FastAPI backend. It implements SDK key authentication, error handling, and provides TypeScript interfaces for every API response.

Key Design Decisions

- Single request() function: All API calls go through one function that handles headers, auth, and errors
- SDK key from localStorage: The X-SDK-Key header is automatically included in every request
- Typed responses: Every endpoint returns a strongly-typed Promise<T>
- ApiError class: Custom error with status code and detail message for UI error handling
- No external dependencies: Uses native fetch API, no axios or similar

Authentication Pattern

```
function getSdkKey(): string {
  if (typeof window === 'undefined') return '';
  return localStorage.getItem('conversiono_sdk_key') || '';
}

// Included in every request:
headers: {
  'Content-Type': 'application/json',
  ...(sdkKey ? { 'X-SDK-Key': sdkKey } : {}),
}
```

API Endpoints Covered

- Sessions: fetchSessions (list+filter), fetchSessionDetail
- Analytics: fetchFunnel, fetchFrictionMap, fetchCohorts, fetchRiskDistribution
- Experiments: fetchExperiments, fetchExperiment, createExperiment, deleteExperiment, fetchExperimentStats
- Interventions: fetchInterventionRecs, fetchInterventionStats
- Merchant: fetchMerchantProfile, fetchUsage, rotateKey

Chapter 3: Custom React Hooks

The useQuery hook in src/lib/hooks.ts implements an SWR-like data fetching pattern without any external dependencies. This keeps the bundle small while providing loading states, error handling, and manual refetch capability.

Hook API

```
interface UseQueryResult<T> {
  data: T | null;
  error: string | null;
  loading: boolean;
  refetch: () => void;
}

function useQuery<T>(
  fetcher: () => Promise<T>,
  deps: unknown[] = []
): UseQueryResult<T>
```

How It Works

- Uses useState for data, error, loading, and a tick counter for refetch
- useEffect triggers the fetcher on mount and when deps or tick change
- Cleanup function sets cancelled=true to prevent state updates on unmounted components
- refetch() increments the tick counter, which triggers a new useEffect cycle

Usage Pattern

```
// Simple fetch on mount:
const { data, error, loading } = useQuery(() => fetchUsage());

// With dependencies (re-fetches when filters change):
const { data } = useQuery(
  useCallback(() => fetchSessions(filters), [filters]),
  [JSON.stringify(filters)]
);
```

Chapter 4: Dashboard Layout & Navigation (CONV-51)

The dashboard uses a shell pattern with a fixed sidebar on desktop and a toggleable mobile drawer. The layout leverages Next.js App Router's nested layouts so the sidebar persists across page navigations without remounting.

Sidebar Component

- 6 navigation items: Overview, Sessions, Analytics, Experiments, Interventions, Settings
- Active state highlighting using usePathname() from next/navigation
- Inline SVG icons (no icon library dependency)
- Exact match for /dashboard, prefix match for sub-routes

Responsive Design

```
// DashboardShell.tsx
// Mobile: sidebar hidden by default, toggle with hamburger
// Desktop (lg+): sidebar always visible

<div className={clsx(
  'fixed inset-y-0 left-0 z-40 lg:static',
  sidebarOpen ? 'translate-x-0' : '-translate-x-full lg:translate-x-0'
)} />

// Overlay backdrop on mobile when sidebar is open
{sidebarOpen && (
  <div className='fixed inset-0 z-30 bg-black/30 lg:hidden'
       onClick={() => setSidebarOpen(false)} />
)}
```

Chapter 5: Overview Dashboard Page

The overview page at /dashboard is the landing page after login. It displays four KPI cards, a conversion funnel bar chart, and a risk distribution pie chart. All data loads in parallel using three separate useQuery hooks.

Data Sources

- fetchUsage() -> KPI cards: total sessions, events, active today, experiments
- fetchFunnel() -> Horizontal bar chart showing session counts per funnel step
- fetchRiskDistribution(5) -> Pie chart with 5 risk buckets color-coded green to red

Chart Implementation

```
// Funnel: horizontal bar chart
<BarChart data={funnel.data.steps} layout='vertical'>
  <XAxis type='number' />
  <YAxis type='category' dataKey='step' />
  <Bar dataKey='sessions' fill='#6366f1' />
</BarChart>

// Risk: Pie chart with Cell-based coloring
<Pie data={buckets} dataKey='session_count' nameKey='range_label'>
  {buckets.map((_, i) => (
    <Cell key={i} fill={RISK_COLORS[i % RISK_COLORS.length]} />
  ))}
</Pie>
```

Chapter 6: Sessions List & Detail (CONV-52, CONV-53)

Sessions List Page

The sessions list at /dashboard/sessions provides a filterable, paginated table of all tracked sessions. Merchants can filter by outcome, device type, minimum risk score, and date range.

- Filter bar: outcome select, device select, risk min input, date from/to
- Table columns: session ID (linked), page URL, started at, outcome badge, risk score, device
- Pagination: previous/next buttons, shows page X of Y and total count
- All filters reset page to 1 when changed (prevents empty pages)

Session Detail Page

The detail page at /dashboard/sessions/[id] shows comprehensive information for a single session, including behavioral features, intervention recommendations, and a scrollable event timeline.

- Breadcrumb navigation back to sessions list
- 4 metric boxes: abandonment risk, friction score, intent label, device type
- Behavioral features grid: 8 ML-computed features with formatted values
- Intervention recommendations: psychology-based suggestions with priority and basis
- Event timeline table: time, type, element, position, velocity (max height 500px, scrollable)

Dynamic Routing

```
// Next.js dynamic route: app/dashboard/sessions/[id]/page.tsx
export default function SessionDetailPage() {
  const { id } = useParams<{ id: string }>();
  const session = useQuery(() => fetchSessionDetail(id), [id]);
  const interventions = useQuery(() => fetchInterventionRecs(id), [id]);
  // Both API calls run in parallel on mount
}
```

Chapter 7: Analytics Dashboard (CONV-54)

The analytics page at /dashboard/analytics is the most data-rich page, combining four visualization types: funnel chart, friction hotspots table, risk distribution histogram, and interactive cohort analysis.

Conversion Funnel

- Vertical bar chart with sessions (indigo) and drop-off (red) bars
- Shows step names on X axis, counts on Y axis

Friction Hotspots

- Scrollable table of top 30 UI elements by friction score
- Columns: element ID, event count, rage clicks, rage click rate, avg hesitation
- Rage click counts highlighted in red for quick identification

Risk Distribution

- Bar chart with 10 configurable buckets
- Shows session count per risk range

Cohort Analysis

- Interactive dimension selector: device_type, country_code, outcome, intent_label
- Combo chart: bars for session count (left Y), line for conversion rate (right Y)
- Re-fetches data when dimension changes using useCallback + dependency array

```
// Cohort dimension switching
const [cohortDim, setCohortDim] = useState('device_type');
const cohorts = useQuery(
  useCallback(() => fetchCohorts(cohortDim), [cohortDim]),
  [cohortDim]
);
```

Chapter 8: Experiments Management (CONV-55)

The experiments page at /dashboard/experiments provides full CRUD for A/B experiments plus statistical analysis. Merchants can create experiments, view results, and get significance testing with recommendations.

Features

- Create form: title, hypothesis, friction type, variant A/B descriptions
- Experiment cards: show title, hypothesis, variants, result badge, conversion delta, sample size
- Stats panel: p-value, significance flag, statistical power, recommendation text
- Delete with confirmation dialog
- Paginated list (20 per page)

Input Validation

```
// HTML5 validation + maxLength for XSS prevention
<input name='title' required maxLength={200} />
<textarea name='hypothesis' maxLength={1000} />
// Backend also validates with Pydantic + sanitize_text()
```

Statistics Display

```
// Stats panel shows computed values from backend Z-test
function StatsPanel({ stats }) {
  return (
    <dl>
      <dt>Conversion Delta</dt><dd>{formatPercent(stats.conversion_delta)}</dd>
      <dt>p-value</dt><dd>{stats.p_value?.toFixed(4)}</dd>
      <dt>Significant?</dt><dd>{stats.is_significant ? 'Yes' : 'No'}</dd>
      <dt>Power</dt><dd>{stats.power?.toFixed(2)}</dd>
    </dl>
    <p>{stats.recommendation}</p>
  );
}
```

Chapter 9: Interventions Dashboard (CONV-55)

The interventions page at /dashboard/interventions visualizes the performance of all psychology-based interventions. It shows a stacked bar chart of outcomes and a detailed statistics table.

Stacked Bar Chart

- X axis: intervention IDs (INT-001 through INT-008)
- Stacked segments: converted (green), dismissed (orange), ignored (gray), bounced (red)
- Gives instant visual feedback on which interventions are effective

Statistics Table

- 8 columns: intervention, triggers, converted, dismissed, ignored, bounced, conv rate, avg delta
- Color-coded values: green for conversions, red for bounced, indigo for conversion rate

Chapter 10: Settings & SDK Key Management

The settings page at /dashboard/settings allows merchants to manage their SDK key and view their merchant profile. The SDK key is stored in localStorage and sent as the X-SDK-Key header with every API request.

SDK Key Management

- Input field shows current key from localStorage
- Save button persists to localStorage and reloads the page
- Rotate button calls POST /merchants/rotate-key with confirmation dialog
- After rotation: new key auto-saved, old key immediately invalidated

Security Considerations

- SDK key stored in localStorage (acceptable for merchant dashboard, not for end-user tokens)
- Key rotation is destructive: confirmation dialog prevents accidental rotation
- Backend uses SHA-256 hashing: raw key never stored server-side

Chapter 11: Reusable UI Component Library

The dashboard includes 6 reusable UI components in src/components/ui/ that provide consistent styling and behavior across all pages.

Component Catalog

- Card / CardHeader / CardBody: Rounded white cards with border, shadow, and optional header
- StatCard: KPI display with label, value, optional subtitle and trend color (up/down/neutral)
- Badge: Pill-shaped status indicator with customizable colors via className
- Spinner: Centered loading animation using Tailwind animate-spin
- ErrorBox: Red-tinted error message with optional retry button
- EmptyState: Centered illustration with title and description for empty data states

Design Principles

- Composition over configuration: Card is split into 3 sub-components
- Tailwind-only: No CSS modules or styled-components
- clsx for conditional classes: Clean ternary-based class merging
- No external icon library: Inline SVGs keep the bundle lean

Chapter 12: Formatting Utilities

The format.ts module provides 7 pure functions for consistent data display across the entire dashboard. All functions handle null/undefined gracefully by returning an em-dash character.

Function Reference

```
formatPercent(0.234)       -> '23.4%'
formatRisk(0.75)           -> '75% High'
formatRisk(0.45)           -> '45% Med'
formatRisk(0.2)            -> '20% Low'
formatDate('2026-03-15T14:30:00Z') -> 'Mar 15, 02:30 PM'
formatDuration(125)        -> '2m 5s'
formatNumber(1234567)      -> '1,234,567'
riskColor(0.8)             -> 'text-red-600'
outcomeColor('purchase')   -> 'bg-green-100 text-green-800'
```

Chapter 13: Build Configuration & Deployment

Tailwind CSS v4 Setup

```
// postcss.config.mjs
const config = {
  plugins: { '@tailwindcss/postcss': {} },
};

// globals.css
@import 'tailwindcss';
```

Build Output

```
Route (app)                              Size      First Load JS
/                                        138 B     87.5 kB
/dashboard                               8.12 kB   207 kB
/dashboard/analytics                     8.68 kB   208 kB
/dashboard/experiments                   5.1 kB    92.4 kB
/dashboard/interventions                 3.13 kB   202 kB
/dashboard/sessions                      4.31 kB   101 kB
/dashboard/sessions/[id]                 4.62 kB   101 kB
/dashboard/settings                      3.81 kB   91.1 kB
```

Environment Variables

```
# .env.local
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1

# Production
NEXT_PUBLIC_API_URL=https://api.conversiono.com/api/v1
```

Chapter 14: Human Intervention Required

The following items require manual human setup or decision-making before the dashboard is fully production-ready:

1. Environment Configuration

- Set NEXT_PUBLIC_API_URL for your deployment environment
- Configure CORS on the backend to allow the frontend origin

2. Authentication Enhancement

- Current auth uses SDK key in localStorage. For production, consider adding proper login flow with JWT tokens and HTTP-only cookies
- Add session timeout and automatic key refresh logic

3. SSL/TLS

- Ensure HTTPS is configured for both frontend and API endpoints
- Set Secure and SameSite flags on any cookies

4. Content Security Policy

- Add CSP headers in next.config.js to prevent XSS
- Configure allowed sources for scripts, styles, and API connections

5. Error Monitoring

- Integrate Sentry or similar for frontend error tracking
- Add performance monitoring (Web Vitals) for production metrics

6. Testing

- Add E2E tests with Playwright or Cypress for critical user flows
- Add component tests with React Testing Library
- Set up visual regression testing for chart components

7. Accessibility

- Run axe-core audit and fix any WCAG 2.1 AA violations
- Add ARIA labels to chart components
- Test keyboard navigation through all pages

8. Data Seeding

- Run the backend seed scripts to populate demo data for testing
- Verify all chart visualizations render correctly with real data